

Python Packaging Essentials

Practical guide that walks you through creating, organizing, testing, and publishing Python packages.

Table of contents

Preface	6
Good to know	6
 I Create a package	 7
Should I create a package?	8
Is it hard?	8
Is it only for large projects?	8
 1 Introduction to packaging	 9
1.1 The <code>__init__.py</code> file	9
1.2 Package vs Module vs Library	9
1.3 Initialize the directory	10
1.4 Reusable Python code	10
1.4.1 Not reusable code	10
1.4.2 Reusable code	10
1.5 Package structure	11
1.5.1 <code>my_module.py</code>	11
1.5.2 <code>__init__.py</code>	11
1.6 Use our package	12
 2 Organize a package	 13
2.1 Add main Python project files	14
2.1.1 <code>pyproject.toml</code>	14
2.1.2 <code>LICENSE</code>	14
2.1.3 <code>.git/</code>	14
2.1.4 <code>.gitignore</code>	15
2.1.5 <code>.venv/</code>	15
2.1.6 <code>README.md</code>	15
2.1.7 <code>sandbox.py</code>	15
2.2 Add an internal function	16
2.3 Final organization	16
 3 Handling dependencies	 22
3.1 Specify dependencies	22
3.1.1 Before	23

3.1.2	After	23
3.2	Avoiding dependencies	23
3.3	Controlling the versions	24
3.3.1	numpy==2.1.0	24
3.3.2	numpy<=2.1.0	25
3.3.3	numpy>=2.1.0,<2.2.0	25
3.3.4	numpy	25
3.3.5	Set the minimum version required...	25
3.3.6	... test your code...	26
3.3.7	... and be convenient	26
3.4	Breaking changes	26
3.5	Required, Optional and Dev dependencies	26
3.5.1	Required	27
3.5.2	Optional dependencies	27
3.5.3	Before	29
3.5.4	After	29
3.5.5	Dev dependencies	29
II	Code quality	31
4	Unit tests	32
4.1	What is unit testing	32
4.1.1	Quick definition	32
4.1.2	Pytest	32
4.2	Example with a simple function	32
4.2.1	Simple function	33
4.2.2	Simple test example and to run it	33
4.3	In real projects	34
4.3.1	Backward compatibility	34
4.3.2	CI	35
4.3.3	Collaborative work and external contributions	35
5	Writing documentation	36
5.1	Goal of documentation	36
5.2	Building a website with zensical	37
5.2.1	Configuration of zensical	37
5.3	Preview our website	38
5.4	Multiple page website	38
5.5	Building reference documentation	39
5.5.1	Docstrings	39
5.5.2	Adding reference documentation in zensical	40
5.6	Advanced configuration	41

5.7	Advanced customization	44
5.8	How to deploy the website	44
6	Errors and warnings	45
7	API design	46
7.0.1	Syntax 1	46
7.0.2	Syntax 2	46
7.0.3	Syntax 3	47
III	Workflow	48
8	Github Actions	49
8.1	TLDR: Github Actions	49
8.2	Unit testing	50
8.3	Create and deploy documentation	53
8.4	Code linting and formatting	55
8.4.1	Add Ruff to your project	55
8.4.2	Create the GitHub Action	56
8.5	Q&A	57
8.5.1	I didn't understand certain things	57
8.5.2	Where does the code run?	57
8.5.3	Can I run it on my machine?	57
9	Pre-commit Hooks	58
9.1	TLDR: What is pre-commit?	58
9.2	How it looks	58
9.3	How to set up	59
9.3.1	1. Configuration	59
9.3.2	2. Install	60
9.3.3	3. Commit	60
9.4	FAQ	61
9.4.1	"Files were modified by this hook"	61
9.4.2	"Hook failed"	61
9.4.3	"Nothing happens when I commit"	61
10	Publish to PyPI	62
10.1	PyPI (Python Package Index)	62
10.2	You don't need PyPI	62
10.2.1	pip install without PyPI	62
10.2.2	Pros	63
10.2.3	Cons	63

10.3	Before distributing	63
10.3.1	Dependencies	63
10.3.2	Security	64
10.4	Distributing to PyPI	64
10.4.1	Build	64
10.4.2	Publishing	65
10.4.3	How to use it	69
IV	Bonus	70
11	Miscellaneous	71
11.1	How to name your package	71
11.1.1	Constraints	71
11.1.2	Best Practices	71
11.2	How to name files	72
11.2.1	Bad file names	72
11.2.2	Good file names	72
11.3	What is the <code>__all__</code> variable	72
12	Just and justfile	74
12.1	What problem it solves	74
12.2	<code>make</code> vs <code>just</code>	74
12.3	Simple example	75
12.4	Why this helps in practice	75
13	Type hinting	76
13.1	What is Python type hinting	76
13.2	What type hints do in practice	76
13.3	What type checkers are	77
13.3.1	Minimal usage	77
13.4	How to create custom types	78
13.4.1	Type aliases (<code>type</code>)	78
13.4.2	<code>NewType</code>	78
13.4.3	<code>TypedDict</code>	79
13.5	Impact for users of your package	79
	Python package template	80
	Contributing	81
	Set up environment	81
	Make changes	82

Preface

The aim of this site is to provide all the must known practices when it comes to create a Python package. It offers **multiple blog posts**, where each of them covers one topic with a few key points. The goal here is to empower anyone with just basic Python knowledge.

We'll go over concrete examples, use clear explanations, and try as much as possible to go straight to the point so that anyone with some Python knowledge can create their own Python packages.

All blog posts aim to be very practical, and do not try to be exhaustive. If you're looking to learn more about a very specific topic, it's recommended to use the official [Python packaging guide](#). As a matter of fact, this previous guide is cited a large number of times in the blog posts.

Good to know

- All blog posts are **independant**. Even if they follow some sort of order, it's perfectly fine to just look at what interests you.
- This site is [open source](#) and in **continuous improvement**. If you want to suggest an improvement (correct an error, improve an explanation, add an example or anything else), it's more than welcomed. It starts [here](#).
- You can download the **PDF** version of this site in the top left of the site / navigation bar.

Part I

Create a package

Should I create a package?

Before starting to create a package, it's good to ask if you should even create one. Because even if it's not fundamentally complicated, it's still some additional work compared to not doing so.

A good rule of thumb for this is to answer the question “Do I plan to reuse the code from this project elsewhere?” If so, then it makes sense to make it a package.

More generally, you can ask yourself whether you're building a tool or using tools. A statistical analysis probably does not belong in a package, while functions to do statistics probably should.

Also, note that it's unlikely you'll regret making a Python project a package, while the opposite might happen more often. Packages, if well done, are usually more modular and better organized compared to non-package projects.

Is it hard?

Not really, especially if you take the time to read about it.

Making a Python package is mostly about organizing your project in a specific, standardized way. There are no low-level computer science concepts that you should know, but rather a more or less large set of rules to respect.

Note that the emphasis here is on best practices for developing high-quality software under the best conditions. Much of what you learn while developing Python packages will also help you become a better programmer in general.

Is it only for large projects?

Not at all! Even if your project is 200 lines of code in a single file, it might make sense to make it a package. You can find a fun example [here](#).

1 Introduction to packaging

Creating a Python package is all about making your code **reusable**, **shareable**, and **easy to install**. Whether you want to publish a library for the world or just organize your own projects better, understanding how packaging works is the first step.

1.1 The `__init__.py` file

At its simplest, a package is just a folder that contains an `__init__.py` file.

```
my_package/  
  __init__.py
```

The presence of `__init__.py` tells Python: “this is a package.”

Even if it’s an empty file, it’s important: it allows you to import parts of your code like this:

```
from my_package import something
```

Without it, Python treats the directory `my_package/` as a regular directory, **not something it can import from**.

Note that this is not always true, but it does not matter here. See [namespace packages](#).

1.2 Package vs Module vs Library

These terms get thrown around a lot. Here’s the quick breakdown:

- **Module:** A single `.py` file (e.g., `my_file.py`)
- **Package:** A directory with an `__init__.py`, possibly containing multiple modules (e.g., multiple files)
- **Library:** A more general programming term and refers to a bundle of code that can be used ([source](#))

Library and package most of the time refer to the same thing. All packages are libraries, the opposite is not true. For the sake of simplicity, **it's ok to consider them “equivalent”**, even though we're mostly interested in packages in practice.

Now, let's create the smallest Python package possible.

1.3 Initialize the directory

The very first step is to create a new directory named *“my_package”*. Inside this directory, create another directory with the same name. The structure should look like this:

```
my_package/  
  my_package/
```

1.4 Reusable Python code

When creating a Python package, we want to write **a reusable piece of code**, not just put in a few scripts that do things. To illustrate:

1.4.1 Not reusable code

```
name = "Joseph"  
message = f"Hello {name}"  
print(message)
```

Hello Joseph

This code does something: it prints a message.

1.4.2 Reusable code

```
def say_hello(name):  
    message = f"Hello {name}"  
    print(message)
```

The code above does ‘nothing’. The only thing it does is create a function object that will be stored in memory. I can now call it and it will execute some code. For example:

```
say_hello("Joseph")
```

Hello Joseph

1.5 Package structure

Now, let's create our first Python module (which is just a file ending in `.py`). We'll call it `my_module.py`.

Our file `my_module.py` will be next to the `__init__.py` we talked about before:

```
my_package/  
  my_package/  
    __init__.py  
    my_module.py
```

1.5.1 `my_module.py`

Listing 1.1 `package_name/my_module.py`

```
def say_hello(name):  
    message = f"Hello {name}"  
    print(message)
```

Here we define the main function of our package.

1.5.2 `__init__.py`

Listing 1.2 `package_name/__init__.py`

```
from my_package.my_module import say_hello  
  
__all__ = ["say_hello"]
```

Without the `__init__.py` file, we wouldn't be able to use `from my_package import say_hello` and would have to use `from my_package.my_module import say_hello`, which isn't the best syntax (but sometimes it can be!).

Note

What is `__all__` variable?

1.6 Use our package

Until now, we were from the package developer side, but how do we use our package, from a user point of view?

For this, you'll need to have [uv](#) installed (not mandatory, but it makes things much easier).

Then we'll need to run a command in our terminal at `Desktop/my_package/`:

```
uv init
uv venv
uv pip install -e .
```

Don't worry too much about those commands yet.

This command will install our current package in **editable mode**. This allows us to test our package while making updates.

The next step is to open a new Python file, console or notebook, ideally not in the package directory. I usually like having a `sandbox.py` file. In this file, we'll run:

```
from package_name import say_hello

say_hello("Julia")
```

Hello Julia

And now we have a **fully functional Python package**! This is just the beginning, but this is an important foundation to have for what's coming next.

Next, we need to [organize the package](#), particularly to get an overview of all the files we need (most of which are not Python files).

2 Organize a package

In order to follow the steps below, you'll need to have both [Git](#) and [uv](#) installed on your machine. Both are command-line tools, meaning you'll use your terminal to run commands that perform various actions.

Let's assume we're naming our Python package “*sunflower*”, with a single function:

```
sunflower/  
  sunflower/  
    __init__.py  
    my_module.py
```

To learn about this structure and why we need a `__init__.py` file, check out the [previous blog post](#).

We'll also assume that `my_module.py` looks like this:

Listing 2.1 `sunflower/my_module.py`

```
import re  
  
def count_sunflowers(s):  
    s = re.sub(r"[^a-zA-Z\s]", "", s) # Remove non-text characters  
    s = s.lower()                    # Convert to lowercase  
    n_sunflower = s.split().count("sunflower")  
    n_sunflowers = s.split().count("sunflowers")  
    return n_sunflower + n_sunflowers
```

The `__init__.py` file look like this:

Listing 2.2 `sunflower/__init__.py`

```
from .my_module import count_sunflowers  
  
__all__ = ["count_sunflowers"]
```

2.1 Add main Python project files

Next, we need to create a few essential files at the root of the project. As you can see, most of them are not Python files, but they are still very important.

2.1.1 `pyproject.toml`

All the metadata for the package. This is essentially your package's identifier, which contains a lot of useful information when we want to distribute a package so that anyone can easily install it.

For example, it contains information about the license (what are users allowed to do with our package?), dependencies (what packages does our package need?), and lots of other metadata about the package (author(s), version, description, etc.).

Here is a simple version of this file:

2.1.2 LICENSE

A basic text file containing the licence for your package. This licence is important because it tells other people what they are allowed to do with your package.

It is specific to each project, but you can find out more at choosealicense.com.

Here is an example of the most common licence: the MIT licence.

2.1.3 `.git/`

This is a directory used internally by the Git software to track all changes in the project. Assuming that the first “*sunflower*” directory is in your `Desktop/` directory, you should create this directory by running the `git init` command when you are in the `Desktop/sunflower/` directory.

It's very likely that **you won't see it**, as most operating systems (Windows, MacOS, etc.) hide files/directories that start with `.`, but it doesn't matter. This directory will be managed entirely by Git itself, so we recommend that you **never make any manual changes to it**.

2.1.4 .gitignore

A file in which each line describes one or more files/directories that are not explicitly part of the project or are not relevant in general. Don't worry too much about this, you can just start with the example content below.

It is very likely that **you will not see it** outside your code editor, as most operating systems (Windows, MacOS, etc.) hide files/directories that start with `.`, but that doesn't matter.

2.1.5 .venv/

A directory containing all the things we need to work properly in our Python environment. It contains a Python interpreter, all the packages used in the project (e.g. `numpy`, `requests`, etc), and a few other things.

The best way to create one is to run `uv venv`.

It is very likely that you will **not** see it outside your code editor, as most operating systems (Windows, MacOS, etc.) hide files/directories that start with `.`, but that doesn't matter.

2.1.6 README.md

A markdown file that describes the project, gives advice on how to use it, install it and so on. There are no rules about what to do with this file, it's just used to tell people what is the first thing they should read before using your package.

For example, it could be something like this:

2.1.7 sandbox.py

A file that we will use to test and use our package. It's optional but practical.

The organisation of our project now looks like this:

```
sunflower/  
  sunflower/  
    __init__.py  
    my_module.py  
  .git/  
  .venv/  
  .gitignore  
  README.md  
  LICENSE
```

```
sandbox.py
pyproject.toml
```

2.2 Add an internal function

When creating a package, it is very useful to create functions that we will use internally: within the package itself, but which are not intended for users of the package.

If we go back to our previous example, we might want to have a separate function that takes a string and cleans it up by removing non-text characters and putting it in lower case. Let's name this function `_clean_string()` and place it in a new file: `other_module.py`.

Our code in `my_module.py` should now become:

We now have 2 functions:

- `count_sunflowers()` a **public** function that users of the package will use.
- `_clean_string()` a **private** function used internally. The underscore ('_') at the beginning of the function name tells other people that it should not be used outside the package from which it came.

Note that `_clean_string()` is still usable by users if they run it:

```
from sunflower.other_module import _clean_string
```

But as you can see from the [documentation blog post](#), we won't have or create documentation on these functions, so they're unlikely to find it anyway.

2.3 Final organization

After all these steps, our package now looks like this:

```
sunflower/
├── sunflower/
│   ├── __init__.py
│   ├── my_module.py
│   └── other_module.py
├── .git/
├── .venv/
├── .gitignore
└── README.md
```



```
LICENSE
sandbox.py
pyproject.toml
```

We now have a basic but fairly robust file organization in our package. Now, if we want to continue improving our package, everything will happen directly in the **sunflower/sunflower** directory.

The next step is to [handle dependencies](#), i.e., identify and specify the packages needed to use our package.

Listing 2.3 pyproject.toml

```
[project]
name = "sunflower"
description = "Create pretty sunflowers"
version = "0.1.0"
license = "MIT"
license-files = ["LICENSE"]
keywords = ["sunflower", "flower"]
authors = [
    { name="your_name", email="your_name@mail.com" },
]
readme = "README.md"
requires-python = ">=3.9"
classifiers = [
    "Programming Language :: Python :: 3",
    "Operating System :: OS Independent",
    "Development Status :: 3 - Alpha"
]
dependencies = []

[build-system]
requires = [
    "setuptools",
    "setuptools-scm",
]
build-backend = "setuptools.build_meta"

[tool.setuptools]
packages = ["sunflower"]

[tool.uv.sources]
sunflower = { workspace = true }

[project.urls]
Homepage = "https://your_name.github.io/sunflower/"
Issues = "https://github.com/your_name/sunflower/issues"
Documentation = "https://your_name.github.io/sunflower/"
Repository = "https://github.com/your_name/sunflower"
```

Listing 2.4 LICENSE

Copyright (c) 2025 Your Name

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Listing 2.5 .gitignore

```
# Python-generated files
__pycache__/
*.py[oc]
build/
dist/
wheels/
*.egg-info

# Virtual environments
.venv/
venv/
.env/
env/

# VS code config
.vscode/

# files on mac
.DS_Store

# all cache files
*cache*

# Sandbox files
sandbox.py
sandbox.ipynb
```

Listing 2.6 README.md

```
# sunflower: my cool Python package

Welcome to the homepage of the `sunflower` project.

It's a new project, but it will be available soon!
```

Listing 2.7 sunflower/other_module.py

```
import re

def _clean_string(s):
    s = re.sub(r"[^a-zA-Z\s]", "", s) # Remove non-text characters
    s = s.lower()                    # Convert to lowercase
    return s
```

Listing 2.8 sunflower/my_module.py

```
from .other_module import _clean_string

def count_sunflowers(s):
    s = _clean_string(s)
    n_sunflower = s.split().count("sunflower")
    n_sunflowers = s.split().count("sunflowers")
    return n_sunflower + n_sunflowers
```

3 Handling dependencies

Dependencies are the external Python packages your code needs in order to work, such as `requests`, `numpy`, or `pandas`.

Here we'll focus on using `uv` to handle dependencies, as it's currently the best tool for this out there (it's fast and fairly easy to use, especially if you know `pip` or other package manager like `Cargo`).

3.1 Specify dependencies

For example, let's say we have this function in our package:

```
import numpy as np

def normalize(array):
    min_val = np.min(array)
    max_val = np.max(array)
    return (array - min_val) / (max_val - min_val)
```

When people want to use our function, they **need** to have `numpy` installed for it to work, otherwise it will raise a `ModuleNotFoundError` on their machine.

So in order to ensure that this does not happen, we need to tell Python to also install `numpy` when installing our package. In practice, we say that we set `numpy` as a **dependency** of our package. This means that every time someone install our package, they will also install `numpy`.

The dependencies of a package are listed in the `pyproject.toml` file. If you don't know what that is, check out [organizing a package](#).

With `uv`, we just have to run:

```
uv add numpy
```

This will automatically add `numpy` to our `pyproject.toml`.

3.1.1 Before

Listing 3.1 pyproject.toml

```
[project]
name = "mypackage"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = []
```

3.1.2 After

Listing 3.2 pyproject.toml

```
[project]
name = "mypackage"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = [
    "numpy>=2.2.4",
]
```

From now on, numpy will also be installed when users install our package. If we add other dependencies, they will be added to the “dependencies” list: we can have as many dependencies as we want, but we want to avoid that.

3.2 Avoiding dependencies

In general, we want to **avoid** having too many dependencies. Why is that? Because when we install a package, we need to install its dependencies too, as well as the dependencies of those packages, and so on.

The issue with this is that it adds a lot complexity quickly and increase the risk of having **conflicts**.

For example, one package might need a version of `numpy` before `<2.0.0`, while another need a version above or equal to `>=2.2.0`. This kind of situation can quickly arise if not careful when adding too many dependencies, and it's usually a nightmare to resolve.

Note: packages with low or no dependencies are called **lightweight**. As an example, have a look at the [narwhals](#) package.

The thing with having lots of dependencies is that it makes it easier for you to write code because you can use other people code super easily. So it's always a trade-off somehow.

Always ask yourself those questions before adding a new package to your dependencies:

- is the dependency a well-known, stable package (`numpy`, `requests`, etc) or is it new and is likely to change in the future?
- does this dependency has lots of dependencies too? This might be a red flag
- can't you just code what it does yourself? If you only need a single function, go check their source code on Github and see if it's easy to do on your side (and ensure their License allow you to copy the code too).

Tip

You can quickly check what are the dependencies of a package with the [Python Dependency Explorer](#)

3.3 Controlling the versions

What's the difference between `numpy 2.1.0` and `numpy 2.0.0`? Well, many things, but for example, in `numpy 2.0.0`, the `np.unstack()` function doesn't exist as it's a new one from `numpy 2.1.0`.

If our package relies on `np.unstack()` in one of our functions, we **can't** let people install any `numpy` version when installing our package. We need to ensure people install this version: `numpy>=2.1.0`. If we translate it, it means *any version of `numpy` above or equal to `2.1.0`*. Let's see some other examples.

3.3.1 `numpy==2.1.0`

Install exactly this version of `numpy`.

```
uv pip install numpy==2.1.0
```


3.3.2 `numpy<=2.1.0`

Install the latest available version before 2.1.0 (including 2.1.0) of `numpy`.

```
uv pip install 'numpy<=2.1.0'
```

3.3.3 `numpy>=2.1.0,<2.2.0`

Install the latest version between 2.1.0 (included) and 2.2.0 (excluded) of `numpy`.

```
uv pip install 'numpy>=2.1.0,<2.2.0'
```

3.3.4 `numpy`

Install the latest version of `numpy`.

```
uv pip install numpy
```

Note that for each of those, the package manager will always try to install the latest version it can depending on the other dependencies. If a package requires `numpy<=2.1.0`, other packages **must** include `numpy 2.1.0` for it to work.

At this point, you might ask, **how do I know** which versions of each dependencies are required for my package? Well, as far as I know, there is no easy answer to this, but there are ways to ensure you don't get unexpected behaviors.

3.3.5 Set the minimum version required...

For each of your dependencies, set in your `pyproject.toml` the minimum version required. With `uv`, you can run the following to install a specific version:

```
uv pip install numpy==2.0.0
```

Warning: the command above will install a specific version of `numpy`, but will not change the requirements in `pyproject.toml`. Use `uv add` if you want to change them.

3.3.6 ... test your code...

You have to test that your code works as expected on those versions. The best way to do that is **unit testing**, and it's the point of the [next blog post](#).

3.3.7 ... and be convenient

Depending on whether you're planning on distributing your package (e.g., put it on PyPI and allow other people to install it) or not, you might want to do different things here. We'll assume you want to distribute it at the end.

When your package goal is to be installed by other people, you want to be convenient. By that I mean **not being too restrictive**.

If we take our example from before, we know that we need at least `numpy==2.1.0` for our package to work, but we also know that **any numpy version above works too**. For this reason, we set `numpy>=2.1.0` instead of `numpy==2.1.0` to allow a broader range of possibility.

3.4 Breaking changes

By default, when installing our package, it will try to find the latest `numpy` version that satisfies the requirements.

But, you might say there's a risk it will break on a new `numpy` version? Yes, **it absolutely does**. And that's exactly why we said earlier why we wanted to avoid having too many dependencies and use stable ones only.

The good thing with packages like `numpy` is that it's one of the most important Python package and one of its core component. They **can't** make breaking changes on any significant feature. When they want to do it, they usually add warnings like this: *"The function xxx is deprecated and will be removed in a future version, please use yyy instead?"*.

But, if you want to be sure you don't get breaking changes, set the maximum version of the dependencies, with things like `numpy<=2.2.0`. This will ensure it's safe, but this also means you'll need to manually update it as new versions come out.

3.5 Required, Optional and Dev dependencies

When working with dependencies, it's useful to differentiate between three main types: required, optional, and development dependencies. Each serves a different purpose in your package.

3.5.1 Required

Required dependencies are the ones we've been discussing so far - packages that your code absolutely needs to function properly. These go in the `dependencies` list in your `pyproject.toml`.

Listing 3.3 `pyproject.toml`

```
[project]
dependencies = [
    "numpy>=2.1.0",
    "pandas>=2.0.0",
]
```

3.5.2 Optional dependencies

Optional dependencies are packages that enhance your code but aren't strictly necessary for core functionality. For example, if your data processing package works with CSV files by default but can also handle Excel files with an additional dependency.

You can specify these in your `pyproject.toml` using the `[project.optional-dependencies]` section:

Listing 3.4 `pyproject.toml`

```
[project.optional-dependencies]
excel = ["openpyxl>=3.1.0"]
plot = ["matplotlib>=3.7.0", "plotly>=5.23.0"]
```

This lets users install only what they need:

3.5.2.1 `mypackage`

Install your package required dependencies **only**:

```
uv pip install mypackage
```

3.5.2.2 mypackage[excel]

Install your package required dependencies as well `openpyxl`:

```
uv pip install "mypackage[excel]"
```

3.5.2.3 mypackage[plot]

Install your package required dependencies as well `matplotlib` and `plotly`:

```
uv pip install "mypackage[plot]"
```

3.5.2.4 mypackage[excel,plot]

Install with all optional dependencies:

```
uv pip install "mypackage[excel,plot]"
```

In your code, you'll need to handle cases where optional dependencies aren't installed:

```
def read_file(filename):
    if filename.endswith('.csv'):
        # Core functionality
        import pandas as pd
        return pd.read_csv(filename)
    elif filename.endswith('.xlsx'):
        try:
            # Optional functionality
            import openpyxl
            import pandas as pd
            return pd.read_excel(filename)
        except ImportError:
            raise ImportError(
                "Excel support requires 'openpyxl'. "
                "Install with 'pip install mypackage[excel]'"
            )
```

! Important

It is **required** here to place `import` inside the function, because otherwise a `ModuleNotFoundError` error will be generated on the user's machine, even when importing a function without optional dependencies.

This will give your users a **clear and meaningful error message** that they can resolve very quickly. This kind of thing exist for the same reason we're talking about in this article: trying to minimize the number of dependencies (especially the unused ones!).

In order to add package to your optional dependencies, you can run:

```
uv add matplotlib --optional plot
```

This will add `matplotlib` to the `plot` section in the optional dependencies in your `pyproject.toml`.

3.5.3 Before

Listing 3.5 `pyproject.toml`

```
[project]
name = "mypackage"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = []
```

3.5.4 After

3.5.5 Dev dependencies

Development dependencies are packages you need only when developing your package, not when using it. These include testing frameworks, documentation generators, linters, and similar tools.

Specify these in your `pyproject.toml` like this:

In order to add a package to your dev dependencies, you can run:

Listing 3.6 `pyproject.toml`

```
[project]
name = "mypackage"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = []

[project.optional-dependencies]
plot = ["matplotlib>=3.7.0"]
```

Listing 3.7 `pyproject.toml`

```
[project.optional-dependencies]
dev = [
    "pytest>=7.0.0",
    "ruff>=0.11.5",
    "sphinx>=7.0.0",
]
```

```
uv add ruff --dev
```

Developers working on your package can install all optional dependencies as well as dev dependencies with:

```
uv sync --all-groups
```

The next section is about `code quality` in your package.

Part II

Code quality

4 Unit tests

This blog post explains the interest of unit testing when developing a Python package. It's **not** meant as an in-depth tutorial about how to do unit tests, but rather an overview of what problem it solves and how it is relevant to Python packaging.

If you want to learn how to do unit testing, the best place to start is the [official documentation](#).

4.1 What is unit testing

Unit testing is the practice of writing small, focused tests to verify that individual parts (units) of your code behave as expected. In the context of a Python package, unit tests help ensure that your functions, classes, and modules continue to work correctly as your code evolves.

4.1.1 Quick definition

A *unit* is the smallest testable part of your application, typically a **single function**. A *unit test* checks if that function produces the correct output for a given input. These tests are automated and are typically run frequently during development.

4.1.2 Pytest

[Pytest](#) is a popular testing framework in the Python ecosystem. It's known for its clean syntax, powerful features (like fixtures and parameterization), and ease of use. Pytest works with simple `assert` statements, so **you don't need to learn a special API to start testing**.

4.2 Example with a simple function

Let's look at how unit tests work using a basic example. We assume that we have a package named `sunflower`:


```
sunflower/  
  sunflower/  
    __init__.py  
  pyproject.toml
```

4.2.1 Simple function

Suppose you have a function that counts how many times the word “sunflower” appears in a string:

Listing 4.1 sunflower/count_sunflower_module.py

```
import re  
  
def count_sunflowers(s):  
    s = re.sub(r"[^a-zA-Z\s]", "", s) # Remove non-text characters  
    s = s.lower() # Convert to lowercase  
    n_sunflower = s.split().count("sunflower")  
    n_sunflowers = s.split().count("sunflowers")  
    return n_sunflower + n_sunflowers
```

4.2.2 Simple test example and to run it

A test is actually a function (or a class) that calls the function we want to test. This new function must

- start with `test_`
- be in a file in the `tests/` directory in a file that starts with `test`

For instance, our project could be organized as follows:

```
sunflower/  
  sunflower/  
    __init__.py  
    count_sunflower_module.py  
  tests/  
    test_count_sunflower.py  
  pyproject.toml
```

Listing 4.2 tests/test_count_sunflower.py

```
import pytest
from sunflower import count_sunflowers

def test_count_sunflowers():
    output = count_sunflowers("sunflower sunflower cake")
    expected = 2
    assert output == expected
```

Our test function will verify if our original function **behaves as expected**. For example, we can test `count_sunflowers()` with the following test:

To run the test, simply use the `pytest` command in the terminal:

```
pytest
```

If the result of `assert output == expected` is `True`, then `pytest` will tell us that everything's fine, otherwise an error message with details on where the error comes from for debugging.

4.3 In real projects

Unit testing becomes even more valuable as your project grows and others start using or contributing to it.

4.3.1 Backward compatibility

Tests protect against accidental changes that could break existing behavior. If you modify a function, running your tests will quickly show if the new version still satisfies the old expectations.

For example, let's say we modify our `count_sunflowers()` function in order to make it 2x faster. **This change should not make anything different** from the user point of view, it should just be faster.

4.3.2 CI

Unit tests are often run automatically using Continuous Integration (CI) tools like GitHub Actions. Whenever someone pushes a commit or opens a pull request, the CI system runs the test suite to make sure nothing is broken. This reinforces trust in the codebase and speeds up development.

There is a [dedicated blog post](#) that covers GitHub Actions in more detail.

4.3.3 Collaborative work and external contributions

Having a robust test suite helps contributors understand what your code is supposed to do. It also encourages better code quality and smoother collaboration, since developers **can make changes confidently** and verify them with tests.

To learn more about how and why to write unit tests, check out the [official documentation](#).

5 Writing documentation

5.1 Goal of documentation

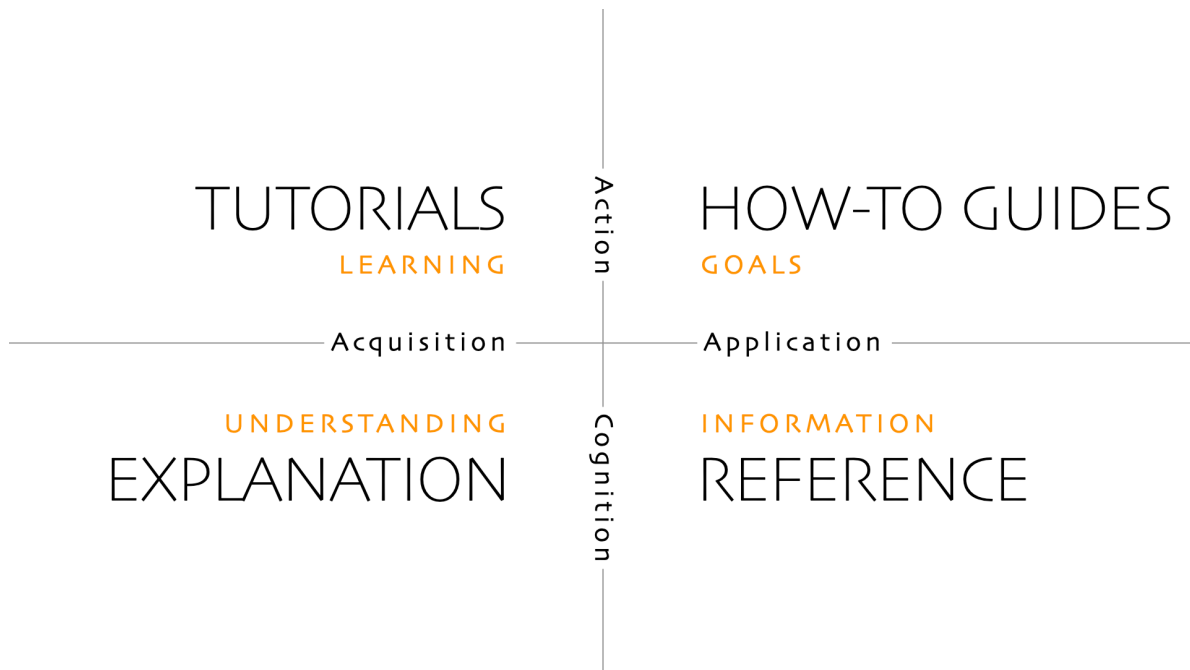
The documentation of a package is the very first, and often only, thing that people will see about your package.

One of the goals of software documentation is to reduce as much as possible the effort required for people to be able to use your project. And for this reason, documentation is as important as the actual code in your package.

Ideally, you want your documentation to be separated into different categories:

- Tutorials: step-by-step introductions with a focus on learning
- How to guides: solve specific, focused problems
- Explanation: deeper context and reasoning about concepts
- Reference: complete, factual list of functions, classes, and arguments in your package

This is entirely based on the [Diátaxis framework](#):



We won't go into detail here about *what to write in the documentation* (that's the role of the [Diátaxis framework](#)), but rather *how to write the documentation, deploy it automatically, and ensure it's always up to date*.

But to give you some concrete and useful advice, here are a few key habits that will help you create better documentation:

- Always start with examples: people understand things much better when they are presented in context rather than in explanations.
- Try to have a narrative: what is your key point? What concept are you trying to explain? Can't we make it simpler?
- Don't focus too much on the “how” (the underlying workings of your package), but rather on the “why” (the concrete problem it solves).

5.2 Building a website with zensical

The very first thing is, your package **needs** a website. Nobody on earth will browse the source code of your project to understand how to use it. Fortunately for us, you need almost zero web development skills to create a documentation website, thanks to tools like [zensical](#) or [sphinx](#).

Here we will use [zensical](#), which is a rewrite of [mkdocs material](#). It allows writing markdown and generating a super clean static¹ website.

Note

Please note that we **don't have to** use [zensical](#) over another documentation framework. Other amazing options such as [sphinx](#), [pdoc](#) or [great-docs](#) (among others) exist.

5.2.1 Configuration of zensical

- First, we install [zensical](#) with:

```
uv add --dev zensical
```

Note

The `--dev` flag tells `uv` to add this dependency to the “dev” dependencies (only required for package developers, not users). For more information on this topic, see the blog post [on handling dependencies](#).

¹A static website is a set of pages that always look the same to everyone (no server side code).

- Then, we create a `zensical.toml` file at the root of the project. The most basic content of it should be:

```
[project]
site_name: Name of your package
```

- Finally, we create a `docs/` directory and a `docs/index.md` file. Write something in markdown in `docs/index.md` such as:

```
## My package

This is the homepage of my super cool package.
```

5.3 Preview our website

Before deploying our website (making it actually visible to other people over the internet), we want to preview it to make sure it looks as expected. This just means serving a local server that shows how it will look on other people's browsers. For this, we run:

```
uv run zensical serve
```

Then, it should display something like this:

```
Serving /Users/josephbarbier/l/my_package/docs on http://localhost:8000
Build started
+ /
```

Open the displayed url (here it is `http://localhost:8000`) in your browser to preview the website.

5.4 Multiple page website

Right now, our website has a single main page. Even though we almost never need hundreds of pages, it makes sense to have many pages, especially to follow the [Diátaxis framework](#) mentioned before.

For instance, let's configure our website to have 3 pages. We change our `zensical.toml` file to this:

```
[project]
site_name: Name of your package
nav = [
    { "Home" = "index.md" },
    { "Examples" = "examples.md" },
    { "About" = "about.md" },
]
```

We also need to add 2 new files (with any kind of markdown content inside): `docs/examples.md` and `docs/about.md`.

`zensical` will automatically create a navigation bar that allows visitors to go to specific pages.

5.5 Building reference documentation

For the moment, we saw how to make a website using plain markdown, but we have not really talked about how to document our code.

Documentation is here to describe what source code does. But the code will inevitably **change over time**, and you want to make sure that documentation, especially “reference” documentation, is always up to date with that.

Reference documentation is the documentation of all classes, functions, and arguments of your package. With `zensical` (and some plugins), we can **automatically** generate pages that will read our source code, collect all arguments, their types, and their docstrings (we will see what that is).

That means every time we change our source code (Python files), our **documentation website changes accordingly**. This is amazing because we do not have to worry about updating this part of the documentation.

5.5.1 Docstrings

The previous is based on parsing **Python docstrings**. Docstrings are triple quote strings that are at the very top of a function definition, and are meant to describe **what a function does** and **what the arguments are supposed to do**. In practice, it looks like this:

```

import re

def count_sunflowers(s: str) -> int:
    """
    Count the total occurrence of the "sunflower" word,
    including both singular and plural cases.

    Arguments:
        s: A string where you want to count occurrences.

    Returns:
        The total occurrence of the word(s) sunflower(s).

    Examples:

        ```python
 from my_package import count_sunflowers

 count_sunflowers("sunflower are beautiful") # outputs 1
 count_sunflowers("what a beautiful day") # outputs 0
 count_sunflowers("sunflowers sunflower") # outputs 2
        ```

    Notes:
        This is case insensitive.
    """
    s = re.sub(r"[^a-zA-Z\s]", "", s)
    s = s.lower()
    n_sunflower = s.split().count("sunflower")
    n_sunflowers = s.split().count("sunflowers")
    return n_sunflower + n_sunflowers

```

Docstrings usually start with an overview of what the function does, and then with a description of the argument(s) and what it returns. It is also relevant to add example usages right after.

5.5.2 Adding reference documentation in zensical

In order to parse our source code, we need to install the `mkdocstrings-python` plugin with:

```
uv add --dev mkdocstrings-python
```


Then, we create a `docs/reference/` directory and our `zensical.toml` becomes:

```
[project]
site_name: Name of your package
nav = [
  { "Home" = "index.md" },
  { "Examples" = "examples.md" },
  { "About" = "about.md" },
  { "Reference" = ["reference/count_sunflowers.md"] },
]

[project.plugins.mkdocstrings.handlers.python]
inventories = ["https://docs.python.org/3/objects.inv"]
```

Inside `docs/reference/count_sunflowers.md`, we put:

```
## Count sunflowers

::: my_package.count_sunflowers
```

Then, when previewing our website, we can see that there is a reference section with the content of our previous docstring, super well formatted.

We do not have to rewrite the name of the arguments and so on thanks to this. The source code becomes the only source of truth in our package, which ensures consistency and reduces a lot of manual work.

Caution

Reference documentation is a very important part of documentation, but it is not exhaustive. It is important that you also add tutorials, guides, and explanations, as suggested in the [Diátaxis framework](#). You can place them in their own directories to allow users to easily navigate your website.

5.6 Advanced configuration

For a large projects, our `zensical` configuration file can be longer in order to add more features! For example the `plotjs` doc configuration file:

```

[project]
site_name = "PlotJS"
repo_url = "https://github.com/y-sunflower/plotjs"
site_url = "https://y-sunflower.github.io/plotjs/"
site_description = "Turn static matplotlib charts into interactive web visualizations"
site_author = "Joseph Barbier"
copyright = ""
Copyright &copy; 2026 Yellow Sunflower
""

extra_css = ["stylesheets/style.css"]
nav = [
    { "Home" = "index.md" },
    { "Gallery" = "gallery/index.md" },
    { "Guides" = [
        "guides/index.md",
        "guides/css/index.md",
        "guides/javascript/index.md",
        "guides/embed-graphs/index.md",
        "guides/troubleshooting/index.md",
        "guides/advanced/index.md",
    ] },
    { "API Reference" = [
        "reference/plotjs.md",
        "reference/css.md",
        "reference/javascript.md",
        "reference/datasets.md",
    ] },
    { "For developers" = [
        "developers/overview.md",
        "developers/parsing-matplotlib-svg.md",
        "developers/svg-parser-reference.md",
        "developers/contributing.md",
    ] },
]

[project.theme]
language = "en"
favicon = "images/favicon.png"
custom_dir = "overrides"
logo = "https://github.com/JosephBARBIERDARNAL/static/blob/main/python-libs/plotjs/image.png"
features = [
    "announce.dismiss",

```

```

"content.code.annotate",
"content.code.copy",
"content.code.select",
"content.footnote.tooltips",
"content.tooltips",
"navigation.footer",
"navigation.indexes",
"navigation.instant",
"navigation.instant.prefetch",
"navigation.path",
"navigation.tabs",
"navigation.sections",
"navigation.top",
"navigation.tracking",
"search.highlight",
"navigation.prune",
]

[[project.theme.palette]]
scheme = "default"
toggle.icon = "lucide/sun"
toggle.name = "Switch to dark mode"

[[project.theme.palette]]
scheme = "slate"
toggle.icon = "lucide/moon"
toggle.name = "Switch to light mode"

[project.theme.icon]
repo = "fontawesome/brands/github"
favicon = "https://github.com/JosephBARBIERDARNAL/static/blob/main/python-libraries/plotjs/favicons/favicon.png"

[project.plugins.mkdocstrings.handlers.python]
inventories = ["https://docs.python.org/3/objects.inv"]
paths = ["."]

[project.plugins.mkdocstrings.handlers.python.options]
docstring_style = "google"
inherited_members = true
show_source = false

```

You can learn more about Zensical features [here](#).

5.7 Advanced customization

So far, we've written some simple Markdown code, but you can change many other things: set a different theme, use pretty user interface elements, add code snippets, and so on.

`zensical` has a ton of super cool customization options, that you can often configure with just one line in your `zensical.toml` file. It is recommended to browse the [official website](#) for this purpose.

If you are interested in a pre made configuration template, check out the [Python package template](#).

5.8 How to deploy the website

The easiest (and recommended way) of deploying your website is to use [Github Actions](#). This is not particularly hard, and everything you need to know about what you can do with it (not only deploy websites) is in a [dedicated blog post](#).

6 Errors and warnings

This page is a work in progress. You can see the current state of the project [here](#).

7 API design

This page is a work in progress. You can see the current state of the project [here](#).

Note that we can use any of the following syntaxes:

7.0.1 Syntax 1

```
from my_package import count_my_package

text = """
my_package petals bright and gold,
my_package fields, a sight to behold.
my_package dreams in the morning light,
Blooming softly, pure and bright.
"""

count_my_package(text)
```

7.0.2 Syntax 2

```
import my_package

text = """
my_package petals bright and gold,
my_package fields, a sight to behold.
my_package dreams in the morning light,
Blooming softly, pure and bright.
"""

my_package.count_my_package(text)
```

7.0.3 Syntax 3

```
import my_package as sfl # or any other alias like "sf" or "sunflo"

text = """
my_package petals bright and gold,
my_package fields, a sight to behold.
my_package dreams in the morning light,
Blooming softly, pure and bright.
"""

sfl.count_my_package(text)
```

Part III

Workflow

8 Github Actions

Having a package implies several things, most importantly:

- creating and deploying a documentation website
- testing that it works as expected using unit tests
- tracking changes with version control (Git)

In this post, we'll walk through 3 essential Github Actions you **need** in your workflow when developing Python packages.

This blog assumes basic Git/Github knowledge (push/pull, pull requests, branches).

8.1 TLDR: Github Actions

Github Actions are scripts that perform tasks (pretty much anything you want) when specific “events” occur. You can do **a lot** with them, but here we'll focus on practical use cases for developing a Python package.

These scripts live in the `.github/workflows/` directory and are written as `yaml` files. For instance, a Python package named “sunflower” with two different Github Actions might be organized like this:

```
sunflower/  
  sunflower/  
    __init__.py  
    module1.py  
    module2.py  
  .github/  
    workflows/  
      unit-tests.yaml  
      code-format.yaml  
  tests/  
  .git/  
  .venv/
```

```
.gitignore
README.md
LICENSE
pyproject.toml
```

On certain “events” (as defined in those scripts), `unit-tests.yaml` and `code-format.yaml` will be triggered.

The events we care about here are:

- Opening a pull request
- Merging or pushing to the main branch

Let’s look at a practical example to understand why these scripts are important.

8.2 Unit testing

If you’re not familiar with unit testing, check out [this dedicated blog post](#).

Suppose we have unit tests written with `pytest` in the `tests/` directory. We can now add a `unit-tests.yaml` file in `.github/workflows/` that looks like this:

```
name: Unit tests

on:
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ${ matrix.os }
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        python-version: ["3.9", "3.13"]

    env:
      UV_PYTHON: ${ matrix.python-version }
    steps:
      - uses: actions/checkout@v4
```

```
- name: Install uv
  uses: astral-sh/setup-uv@v5

- name: Enable caching
  uses: astral-sh/setup-uv@v5
  with:
    enable-cache: true

- name: Install the project
  run: uv sync --all-groups

- name: Run tests
  run: uv run pytest
```

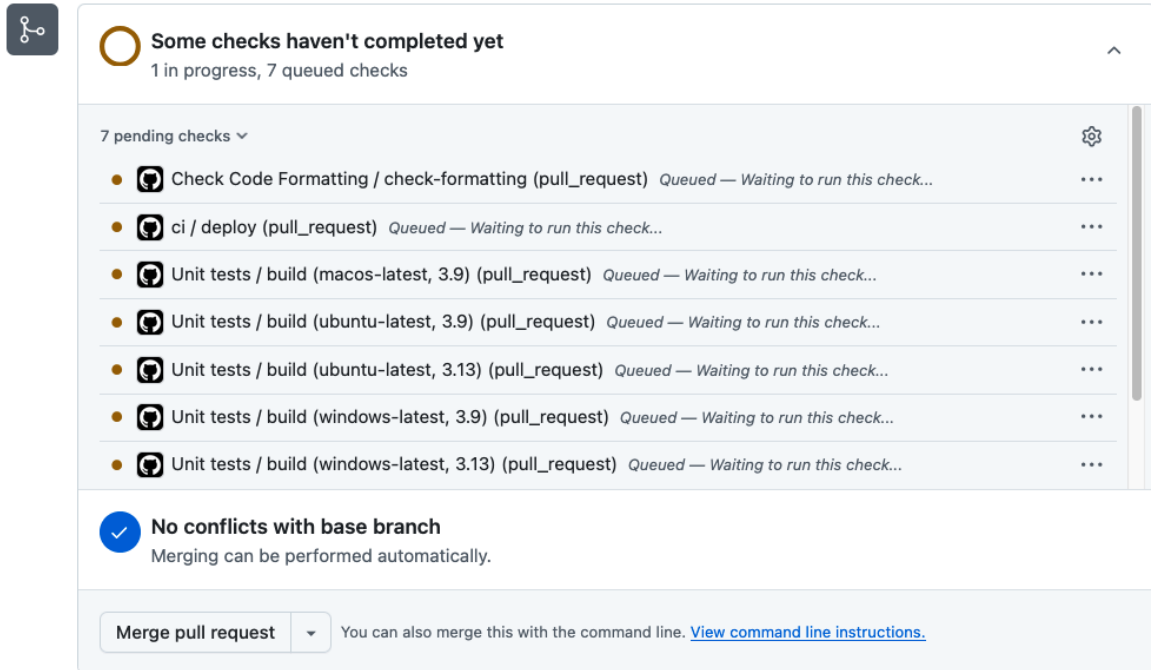
What it does:

When someone pushes a commit to a branch with an open pull request (as specified in the **on** section), this action will:

- install **uv** (a Python package manager)
- install the project dependencies using **uv sync --all-groups**
- run the test suite with **uv run pytest**

This runs across multiple Python versions (3.9 and 3.13) and operating systems (Windows, macOS, and Linux). That gives us 6 combinations in total (2 Python versions $\times$ 3 OSes).

Here's what shows up on the pull request while the tests are running:



The image shows a GitHub pull request status interface. At the top, a yellow circle icon is followed by the text "Some checks haven't completed yet" and "1 in progress, 7 queued checks". Below this, a section titled "7 pending checks" contains a list of seven checks, each with a yellow circle icon, a GitHub logo, a check name, a status, and a three-dot menu. The checks are: "Check Code Formatting / check-formatting (pull_request)", "ci / deploy (pull_request)", "Unit tests / build (macos-latest, 3.9) (pull_request)", "Unit tests / build (ubuntu-latest, 3.9) (pull_request)", "Unit tests / build (ubuntu-latest, 3.13) (pull_request)", "Unit tests / build (windows-latest, 3.9) (pull_request)", and "Unit tests / build (windows-latest, 3.13) (pull_request)". All checks have a status of "Queued — Waiting to run this check...". Below the list, a blue circle icon with a checkmark is followed by the text "No conflicts with base branch" and "Merging can be performed automatically.". At the bottom, there is a button labeled "Merge pull request" and a link to "View command line instructions."

Some checks haven't completed yet
1 in progress, 7 queued checks

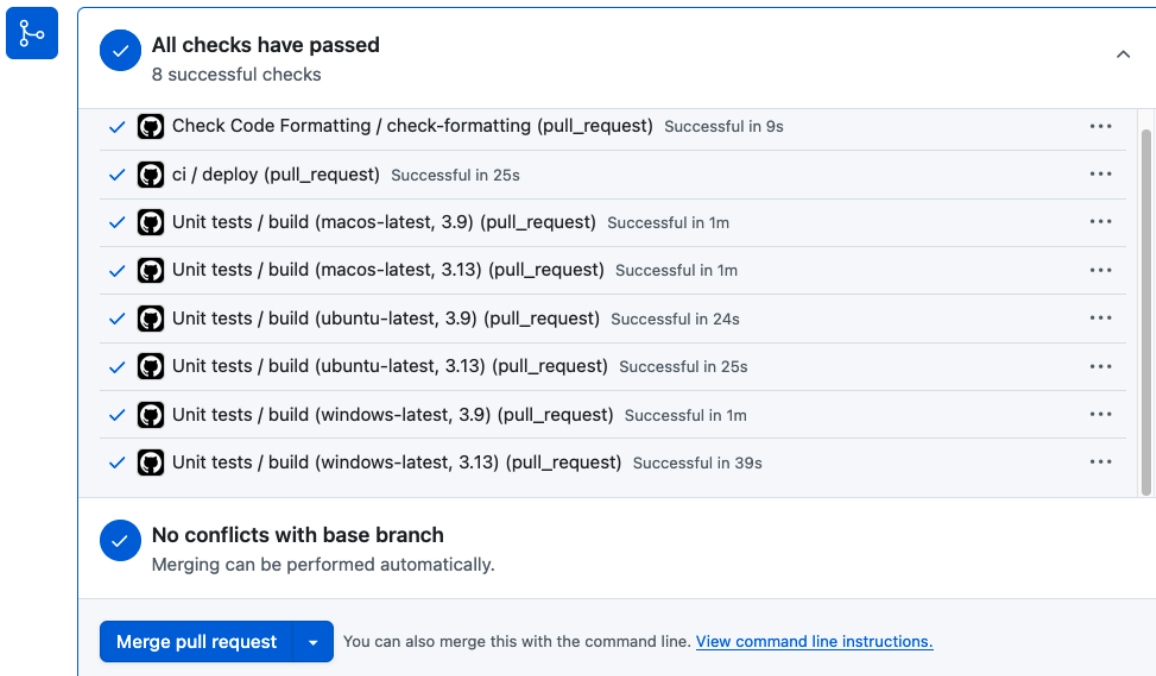
7 pending checks

- Check Code Formatting / check-formatting (pull_request) Queued — Waiting to run this check...
- ci / deploy (pull_request) Queued — Waiting to run this check...
- Unit tests / build (macos-latest, 3.9) (pull_request) Queued — Waiting to run this check...
- Unit tests / build (ubuntu-latest, 3.9) (pull_request) Queued — Waiting to run this check...
- Unit tests / build (ubuntu-latest, 3.13) (pull_request) Queued — Waiting to run this check...
- Unit tests / build (windows-latest, 3.9) (pull_request) Queued — Waiting to run this check...
- Unit tests / build (windows-latest, 3.13) (pull_request) Queued — Waiting to run this check...

No conflicts with base branch
Merging can be performed automatically.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

If any of those combinations fail (meaning at least one test fails), you'll see a message indicating that something didn't work. For example, you might learn that your package fails on Windows with Python 3.9. Otherwise, you'll see something like this:



The purpose of setting up this Github Action is to **automatically and easily** verify that the package works in different environments, helping ensure that only valid code is merged into the main branch.

In this example, we used just two Python versions and one set of dependencies, but this approach can be extended to test the package under many more scenarios. That way, we get a clear and precise picture of what works and what doesn't.

8.3 Create and deploy documentation

There's a dedicated blog post on how to write documentation for your package. Check it out [here](#).

Let's say we've created our documentation website with `mkdocs`. We then add a `deploy-site.yaml` file in `.github/workflows/`.

Since generating the documentation website creates a large number of files, it's not ideal to store them in version control. But how do we deploy it to Github Pages if it's not in version control? That's where Github Actions come in!

Now, let's take a look at the following Github Action script:

```

name: ci

on:
  push:
    branches: [main]

permissions:
  contents: write

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Configure Git Credentials
        run: |
          git config user.name github-actions[bot]
          git config user.email 41898282+github-actions[bot]@users.noreply.github.com
      - uses: actions/setup-python@v5
        with:
          python-version: 3.x
      - run: echo "cache_id=$(date --utc '+%V')" >> $GITHUB_ENV
      - uses: actions/cache@v4
        with:
          key: mkdocs-material-${{ env.cache_id }}
          path: .cache
          restore-keys: |
            mkdocs-material-

      - name: Install uv
        uses: astral-sh/setup-uv@v5

      - name: Enable caching
        uses: astral-sh/setup-uv@v5
        with:
          enable-cache: true

      - name: Install the project
        run: uv sync --all-groups

      - name: Deploy MkDocs
        env:

```

```
GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }}  
run: uv run mkdocs gh-deploy --force
```

What it does:

When someone merges or pushes to the main branch, this action will:

- install `uv` (a Python package manager)
- install the project dependencies using `uv sync --all-groups`
- generate the entire documentation website with `uv run mkdocs gh-deploy --force`
- push the documentation website to the `gh-pages` branch on Github

With this setup, assuming our website is deployed to Github Pages using the `gh-pages` branch, the documentation site is deployed automatically whenever a pull request is opened or we merge/push to the main branch. All without keeping the auto-generated files in version control.

This also **removes all manual work** related to building and deploying the documentation, as it's now fully automated through this Github Action.

8.4 Code linting and formatting

When working on a project, it's crucial to maintain standardized coding practices:

- Consistent formatting (e.g., indentation, spacing, quotes)
- Clean code free of unused imports, bad patterns, or minor bugs

This is where **code formatting** and **linting** tools come into play, and we can automate them using Github Actions.

We'll use `ruff` here, which is a super fast linter and formatter for Python. It can both check for issues (like `flake8` or `pylint`) **and** format code (like `black`), all in one tool.

8.4.1 Add Ruff to your project

First, add Ruff as a development dependency:

```
uv add --dev ruff
```

8.4.2 Create the GitHub Action

Now, create a file named `.github/workflows/code-format.yaml` with the following content:

```
name: Ruff lint and format

on:
  pull_request:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install uv
        uses: astral-sh/setup-uv@v5

      - name: Enable caching
        uses: astral-sh/setup-uv@v5
        with:
          enable-cache: true

      - name: Install dependencies
        run: uv sync --all-groups

      - name: Check formatting
        run: uv run ruff format . --check

      - name: Lint code
        run: uv run ruff check .
```

What it does:

When a pull request is opened against `main`, this action will:

- Install your project and its development dependencies using `uv`

- Check if the code is properly formatted with `ruff format . --check`
- Lint the code with `ruff check .` to catch any potential issues

If there's anything wrong (such as a file needing formatting or an unused import), the action will fail, and the pull request will show a red `.` That's your cue to fix the code.

This ensures that all code added to the codebase is well-formatted and adheres to the established rules.

Note that there's an additional way to enforce this called `pre-commit`, and there's a [dedicated blog post](#) on it.

8.5 Q&A

8.5.1 I didn't understand certain things

Github Actions is one of those things where you need to try it yourself to get the full picture. I recommend creating a basic Python package with documentation and tests, then testing the examples provided to see how they work.

8.5.2 Where does the code run?

When a Github Action is triggered, Github sets up a clean VM (virtual machine) to run your workflow. There are limits on usage, but they're quite generous before you'll need to enter your credit card details.

8.5.3 Can I run it on my machine?

Yes, you can! Thanks to a project called [act](#). In short, it uses [Docker](#) to run your Github Actions in the correct context.

9 Pre-commit Hooks

In this post, we'll walk through how to add `pre-commit` to your Python package to enforce good code hygiene *automatically*.

This post assumes you're already using Git and are familiar with what commits are. It also requires using [uv](#).

9.1 TLDR: What is `pre-commit`?

`pre-commit` is a framework for managing and running “hooks”, which are just scripts defined in a `.pre-commit-config.yaml` file that run at specific points in the Git lifecycle. The most common is the `pre-commit` hook, which runs **before** a commit is created.

The point of using pre-commit hooks is to prevent your codebase from including unwanted things, such as unformatted code, oversized files, print statements, and so on.

Here's what makes `pre-commit` awesome:

- **It runs locally** – unlike CI (e.g. [GitHub Actions](#)), it catches issues *before* they get pushed
- **It's fast** – runs only on the files you've changed
- **It's customizable** – tons of hooks are available, or you can write your own
- **It integrates with CI** – you can run `pre-commit` in CI to make sure everyone follows the same rules

9.2 How it looks

Here is a pre-commit hook that:

- checks if our code is both linted and formatted
- if not, it will try to fix it

```

repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.11.7
  hooks:
    - id: ruff
      types_or: [python, pyi]
      args: [--fix]
    - id: ruff-format
      types_or: [python, pyi]

```

This will run each time we run `git commit`. If our code is not perfectly linted and formatted, it will prevent the commit and lint/format it (if possible).

Once our code is fixed, we can re-run `git add` and `git commit`, and it will accept our commit, which we can then push.

9.3 How to set up

9.3.1 1. Configuration

Create a `.pre-commit-config.yaml` file at the root of your project.

Let's use a relatively common pre-commit setup. It configures formatting and linting with [ruff](#), checks for large files, and removes trailing whitespace:

```

repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.11.7
  hooks:
    - id: ruff
      types_or: [python, pyi]
      args: [--fix]
    - id: ruff-format
      types_or: [python, pyi]

- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v5.0.0
  hooks:
    - id: trailing-whitespace
    - id: check-added-large-files

```

- The `repo` field defines where the actual checks to run are defined
- Each `rev` pin ensures you're using a specific version of that hook for reproducibility
- The `id` defines the exact check to run

9.3.2 2. Install

Install it as a development dependency:

```
uv add --dev pre-commit
uv run pre-commit install
```

If you don't want to add it to your development dependencies, you can simply run :

```
uv pip install pre-commit
uv run pre-commit install
```

9.3.3 3. Commit

Now try editing a Python file (add spaces at the end of the file or change the formatting of something). Then try committing it:

```
git add .pre-commit-config.yaml
git commit -m "Test pre-commit"
```

You'll see the hooks run automatically and fix (or block) your commit if needed.

```
[INFO] Installing environment for https://github.com/astral-sh/ruff-pre-commit.
[INFO] Once installed, this environment will be reused.
[INFO] Running ruff-format...
[INFO] Files were modified by this hook. Please stage the changes and try again.
```

If `pre-commit` fixes files automatically, it will ask you to re-stage them and try committing again. This prevents broken or messy code from slipping into version control.

9.4 FAQ

9.4.1 “Files were modified by this hook”

This means the hook fixed your files. Run:

```
git add -A  
git commit -m "message"
```

9.4.2 “Hook failed”

Some hooks (like `ruff`) don’t fix issues automatically. You’ll need to fix them manually based on the error messages.

9.4.3 “Nothing happens when I commit”

Did you run `uv run pre-commit install`? That installs the Git hook. Without it, the hooks won’t run.

10 Publish to PyPI

This blog post is a work in progress.

10.1 PyPI (Python Package Index)

[PyPI](#) is the default online repository for Python packages. This is where packages are stored so that others can find and install them.

When you run:

```
pip install requests
```

You're downloading the `requests` package from PyPI. More specifically, you're downloading the package's distribution (which might be source code or precompiled binaries) to your local machine from PyPI servers.

10.2 You don't need PyPI

If you're here to learn how to publish a Python package to PyPI, **you're in the right place**.

But it's important that you become aware of the following facts:

- you don't need PyPI to allow anyone in the world to `pip install` your package
- adding your package to PyPI takes a namespace name forever ([related and interesting blog post](#))
- it's a bit complicated to upload your package the clean way

10.2.1 `pip install` without PyPI

If your package is on Github/Gitlab (it should) and is open source, then people can super easily install it with:

```
pip install git+https://github.com/github_username/repository_name.git
```

10.2.2 Pros

- No extra work
- Updates automatically when you push changes

10.2.3 Cons

- Managing dependencies is harder (for users)
- You don't have a clean `pip install package_name`

The main reason behind using PyPI is that it simplifies a lot package versionning (installing a specific version of a package) and you're sure of what you're getting, while a Github repo might just disappear one day.

If your project is just a toy project, with no clear versionning of and actual maintenance, you probably shouldn't put it on PyPI.

10.3 Before distributing

Before putting your package on PyPI, we need to make sure it checks all the valid boxes.

10.3.1 Dependencies

i Note

If you haven't already, check out the [handling dependencies blog post](#), which is an important pre-requisite.

Everything that is inside your `pyproject.toml` defines what will be used for your package once on PyPI. Fields like `name` or `author` will be displayed on the PyPI page (example of the [PyPI page of scikit-learn](#)).

But more importantly, the dependencies and Python version required will be used from this file. If you don't know if things are missing/invalid from your `pyproject.toml`, check out [the official guide](#).

10.3.2 Security

When distributing your package, security is critical. Since an installed package can execute arbitrary code, everything that we decide to put on PyPI should be clear.

Rules to follow absolutely:

- Keep source in sync: The code on PyPI should match the code on GitHub (or any other public repo). Users trust that what they download is the same as what’s publicly visible. Avoid “hidden” changes that only exist in the PyPI release.
- Strong authentication: Use a strong, unique password for your PyPI account and enable two-factor authentication (2FA). PyPI now requires 2FA for maintainers of critical projects, and it’s a good habit for all packages. Avoid relying on “trusted publisher” features until you fully understand them.

10.4 Distributing to PyPI

Distributing to PyPI means multiple things:

- building your package
- uploading that build to PyPI servers

10.4.1 Build

When we talk about *building a package*, we mean taking your source code and metadata (from `pyproject.toml`, `README.md`, etc.) and transforming them into **distribution artifacts**.

There are two main types of distributions:

- **Source distribution (sdist)** → a tarball (`.tar.gz`) containing your source code.
- **Built distribution (wheel)** → a pre-built archive (`.whl`) that can be installed faster because it doesn’t require building on the user’s machine.

Having a proper build step matters because:

- it ensures reproducibility (everyone installing your package gets the same artifact)
- it avoids shipping incomplete or broken files
- it decouples packaging from distribution (you can build once and upload anywhere)

In practice, we don’t want to build the package manually, and for this we’ll use **trust publishing**.

10.4.2 Publishing

Once you have your built artifacts, the next step is uploading them to PyPI. Historically (and probably still a lot today), this has been done with [twine](#):

```
twine upload dist/*
```

This is still valid, but it requires you to handle credentials (API tokens) locally.

The modern, recommended approach is to use **trusted publishing** — where PyPI integrates directly with GitHub Actions, GitLab CI, or other CI/CD providers. This way, you don't store secrets at all, and only builds coming from your repository can publish under your package name.

Here we'll use Github Actions (check out the [dedicated blog post](#)), and in particular the following script:

```
# This is taken from
# https://packaging.python.org/en/latest/guides/publishing-package-distribution-releases-using-github-actions-ci-cd-workflows/
# but with the following differences
# - removed the TestPyPI part
# - instead of `on: push`, we have `tags` in there too

name: Publish Python distribution to PyPI

on:
  push:
    tags:
      - "v[0-9]+.[0-9]+.[0-9]+"

jobs:
  build:
    name: Build distribution
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4
        with:
          persist-credentials: false
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.x"
```

```

- name: Install pypa/build
  run: python3 -m pip install build --user
- name: Build a binary wheel and a source tarball
  run: python3 -m build
- name: Store the distribution packages
  uses: actions/upload-artifact@v4
  with:
    name: python-package-distributions
    path: dist/

publish-to-pypi:
  name: >-
    Publish Python  distribution  to PyPI
  if: startsWith(github.ref, 'refs/tags/') # only publish to PyPI on tag pushes
  needs:
    - build
  runs-on: ubuntu-latest
  environment:
    name: pypi
    url: https://pypi.org/p/pytest-cov
  permissions:
    id-token: write # IMPORTANT: mandatory for trusted publishing

  steps:
    - name: Download all the dists
      uses: actions/download-artifact@v4
      with:
        name: python-package-distributions
        path: dist/
    - name: Publish distribution  to PyPI
      uses: pypa/gh-action-pypi-publish@release/v1

github-release:
  name: >-
    Sign the Python  distribution  with Sigstore
    and upload them to GitHub Release
  needs:
    - publish-to-pypi
  runs-on: ubuntu-latest
  permissions:
    contents: write # IMPORTANT: mandatory for making GitHub Releases
    id-token: write # IMPORTANT: mandatory for sigstore

```

```

steps:
  - name: Download all the dists
    uses: actions/download-artifact@v4
    with:
      name: python-package-distributions
      path: dist/
  - name: Sign the dists with Sigstore
    uses: sigstore/gh-action-sigstore-python@v3.0.0
    with:
      inputs: >-
        ./dist/*.tar.gz
        ./dist/*.whl
  - name: Create GitHub Release
    env:
      GITHUB_TOKEN: ${GITHUB_TOKEN}
    run: >-
      gh release create
      "$GITHUB_REF_NAME"
      --repo "$GITHUB_REPOSITORY"
      --notes ""
  - name: Upload artifact signatures to GitHub Release
    env:
      GITHUB_TOKEN: ${GITHUB_TOKEN}
    # Upload to GitHub Release using the `gh` CLI.
    # `dist/` contains the built packages, and the
    # sigstore-produced signatures and certificates.
    run: >-
      gh release upload
      "$GITHUB_REF_NAME" dist/**
      --repo "$GITHUB_REPOSITORY"

```

What happens in the workflow? Once a [tag](#) is pushed:

- **Build job:** GitHub Actions checks out your repo, installs `build`, and creates both the wheel (`.whl`) and the source distribution (`.tar.gz`).
- **Publish job:** Those artifacts are uploaded directly to PyPI using trusted publishing (no API tokens needed).
- **GitHub Release job:** The distributions are signed with [Sigstore](#) and attached to a GitHub Release, so users can download the same artifacts outside PyPI if they prefer.

! Important

For the above script to work, you first need to [add a publisher in PyPI](#), and this for each package you have.

The best way to use the GitHub Actions workflow above is to pair it with a small release script like this:

```
#!/bin/bash

# Usage: ./release.sh 1.2.3
# Assumes version is set in pyproject.toml or setup.cfg

set -e

VERSION=$1

if [[ -z "$VERSION" ]]; then
    echo " Error: No version number supplied."
    echo " Usage: ./release.sh 1.2.3"
    exit 1
fi

TAG="v$VERSION"

# Confirm version bump is in code
echo " Preparing release: $TAG"
grep "$VERSION" pyproject.toml || {
    echo " Version $VERSION not found in pyproject.toml. Did you forget to bump it?"
    exit 1
}

# Commit the version bump
git add -A
git commit -m "Release $TAG"

# Create tag
git tag "$TAG"

# Push commit and tag
git push origin main
git push origin "$TAG"
```

```
echo " Release $TAG pushed! GitHub Actions will handle the rest."
```

Save this script as `release.sh` (for example), make it executable (`chmod +x release.sh`), and keep it at the root of your repository.

10.4.3 How to use it

1. Bump the version in `pyproject.toml`.

2. Run:

```
./release.sh 1.2.3
```

3. The script will:

- check the version string exists in `pyproject.toml`
- commit the change
- tag it as `v1.2.3`
- push both the commit and the tag

This way:

- you don't need to manually run `python -m build` or `twine upload`
- you don't need to manage PyPI tokens or secrets
- releases are reproducible and cryptographically signed
- the release process becomes a single command:

```
./release.sh 1.2.3
```

Part IV

Bonus

11 Miscellaneous

11.1 How to name your package

Yes, creating a good Python package name is both an art and a bit of a science. There **are constraints** you should follow, and some **best practices** that can help your package stand out and be easy to use.

11.1.1 Constraints

- **Lowercase only:** Package names should be all lowercase.
- **No special characters or spaces:** Use only letters, numbers, and underscores or dashes (a-z, 0-9, _, -).
- **Can't conflict with standard library modules:** Avoid names like `json`, `os`, `email`, etc.
- **Must be unique on PyPI:** Check if the name is available: <https://pypi.org/>
- **Max length:** There's no strict limit, but practical limits (about 50 characters) make sense.
- **Underscores vs Dashes:**
 - Use dashes (-) in the **distribution name** (`setup.py` or `pyproject.toml`).
 - Use underscores (_) or no separator at all in **importable module names**.

11.1.2 Best Practices

- **Short & memorable:** Easier for users to type and remember.
- **Descriptive but concise:** Reflect what the package does.
 - Good: `requests`, `black`, `httpx`
 - Bad: `jdghfc`, `my_cool_package`, `python_toolkit_2023_version_final`
- **Avoid generic terms** unless paired cleverly: `data`, `utils`, `tools`, etc.
- **Avoid abbreviations** unless well-known.
- **Check for conflicts:** Google the name and check on GitHub too, not just PyPI.
- **Consider branding:** If it becomes popular, the name matters.

11.2 How to name files

Having good filenames is mostly useful for having a clear and consistent project architecture. There are some best practices to follow:

- use lowercase only
- avoid spaces and odd characters
- keep it short
- use underscores “_”

11.2.1 Bad file names

```
my file.py
Myfile.py
myFile.py
my@file.py
my-file.py
this-file-does-this-and-that.py
```

11.2.2 Good file names

```
my_file.py
myfile.py
```

11.3 What is the `__all__` variable

The `__all__` variable in Python lives in the `__init__.py` file and is used to control **what gets imported** when someone uses `from my_package import *`. It should be defined at the module level as a list of strings, where each string is the name of a symbol—like a function, class, or variable—that you want to make publicly available.

If `__all__` is present, only those names listed will be imported during wildcard imports. If it's not defined, Python will import all names that don't start with an underscore by default.

For example, consider a file called `my_module.py`:

And our `__init__.py` file:

Now if a user does `from my_package import *`, only `cool_function` will be available. Attempting to use `CoolClass` will raise a `NameError`.

Listing 11.1 my_package/my_module.py

```
def cool_function():  
    return "This is public"  
  
class CoolClass:  
    pass
```

Listing 11.2 my_package/__init__.py

```
from .my_module import cool_function, CoolClass  
  
__all__ = ["cool_function"]
```

It's also important to have in mind that, most of the time, it's highly discouraged to use `from my_package import *` as it does not explicit what is actually imported. Interestingly, it's even [not allowed in marimo notebooks](#).

12 Just and justfile

`just` is a command runner. A `justfile` is where you define reusable commands (called recipes) for your project.

The main goal is simple: avoid remembering long terminal commands, and keep common development tasks in one place.

12.1 What problem it solves

When working on a package, we run the same things again and again:

- preview documentation
- run tests
- run test coverage
- run scripts

Without a command runner, everyone on the project may run slightly different commands. Over time, this creates friction and mistakes.

A `justfile` gives a shared set of commands with clear names like `just docs` or `just test`.

12.2 make vs just

`make` has been used for this for a long time, and it still works very well.

`just` changes the developer experience:

- syntax is simpler for task running
- command names are easier to read
- less boilerplate for common workflows
- designed for command recipes (not only build dependency graphs)

So for Python projects where we mostly want clean task shortcuts, `just` often feels more direct.

12.3 Simple example

Install just from the [official website](#), then create a `justfile` at the root of your project:

```
# Preview docs locally
docs:
    uv run zensical serve

# Run tests
test:
    uv run pytest

# Run tests with coverage
coverage:
    uv run pytest --cov=my_package --cov-report=term-missing

# Run one Python script
script:
    uv run python scripts/example.py
```

Then you can run:

```
just docs
just test
just coverage
just script
```

12.4 Why this helps in practice

A small `justfile` gives your project a single command entrypoint. New contributors do not need to guess how to run things, and CI/local commands are easier to keep aligned.

If you want details on the tasks themselves, see:

- [Writing documentation](#)
- [Unit tests](#)

13 Type hinting

This blog post explains what type hinting is in the context of Python package development, what it changes in practice, and how tools like [ty](#) help you and your users catch issues before runtime.

13.1 What is Python type hinting

Type hinting means annotating function arguments, return values, and variables with expected types.

Listing 13.1 `sunflower/my_module.py`

```
def count_sunflowers(text: str) -> int:
    return text.lower().split().count("sunflower")
```

In this example:

- `text: str` says the function expects a string for the `text` argument.
- `-> int` says the function returns an integer.

Type hints make your API easier to understand and document, especially when your package grows and other people contribute.

13.2 What type hints do in practice

Type hints are mostly metadata unless you use external tools.

According to the Python docs, runtime Python does not enforce annotations by default. That means:

- Python will not automatically reject wrong argument types just because you added annotations.

```
a: int = "a" # This does not raise an error
```

- Adding annotations does not make your code faster.
- Type checking happens with **separate tools** called type checkers, not with the interpreter itself.

So the value of type hints comes from static analysis, better editor support, and clearer package APIs.

13.3 What type checkers are

A type checker reads your code and verifies that values flow through your program in ways that match the annotations.

A modern option is `ty`, from Astral (the team behind `uv` and `ruff`).

13.3.1 Minimal usage

From a project directory:

```
uvx ty check
```

Or if `ty` is installed in your environment:

```
ty check
```

Useful day-to-day commands:

```
ty check --watch
```

`ty` can also be configured in `pyproject.toml` under `[tool.ty]`.

In package development, this usually becomes part of your quality workflow, just like tests and linters.

When developing a package, you'll likely want to add it inside your code editor (VS Code, Vim, etc). This will make your life easier and catch type issues as soon as they arrive.

Listing 13.2 `pyproject.toml`

```
[tool.py.rules]
all = "warn"
possibly-unresolved-reference = "error"

[tool.py.terminal]
error-on-warning = true
```

13.4 How to create custom types

Custom types are useful when your package has domain concepts that should be explicit.

13.4.1 Type aliases (type)

Great for readability when a shape is used repeatedly.

```
type UserRecord = dict[str, str | int]

def format_user(user: UserRecord) -> str:
    return f"{user['name']} ({user['id']})"
```

13.4.2 NewType

Useful when two values share the same runtime type but represent different concepts.

```
from typing import NewType

UserId = NewType("UserId", int)
OrderId = NewType("OrderId", int)

def get_user(user_id: UserId) -> str:
    return f"user-{user_id}"
```

A type checker can catch passing an `OrderId` where a `UserId` is expected.

13.4.3 TypedDict

Useful when your package accepts structured dictionaries.

```
from typing import TypedDict

class PackageMeta(TypedDict):
    name: str
    version: str

def describe(meta: PackageMeta) -> str:
    return f"{meta['name']}=={meta['version']}"
```

This documents required keys and helps catch missing or invalid fields.

13.5 Impact for users of your package

Type hints improve the experience of users who run a type checker (`ty`, `mypy`, `pyright`, etc.) in their own projects.

When your package is typed well:

- users get earlier feedback in their editor/CI before runtime,
- function signatures are clearer,
- integration mistakes are caught sooner,
- refactoring is safer on both your side and the user side.

For package maintainers, this means fewer avoidable bug reports caused by obvious misuse, and better API contracts over time.

If you want to go deeper, the best references are:

- Python typing docs: [typing — Support for type hints](#)
- Astral `ty` docs: docs.astral.sh/ty

Python package template

As long as the website you're currently at, we made a **Python package template**. It's basically a Github repo with everything pre-defined: metadata, documentation, CI/CD, pre-commit, etc.

You can find it [here](#).

It relies almost entirely on all the principles described on this website, which are primarily designed to enable users to quickly start working in a clean environment for their package.

Contributing

Set up environment

In order to follow the steps below, you'll need to have [Git](#), [uv](#) and [Quarto](#) installed on your machine.

- Fork the [Github repo](#)
- Git clone it:

```
git clone https://github.com/YourUsername/python-packaging-essentials.git
```

- Create a new Git branch

```
git checkout -b branch-name
```

- Set up your environment

13.5.0.1 MacOS/Unix-like

```
uv python install
uv sync
source .venv/bin/activate
```

13.5.0.2 Windows

```
uv python install
uv sync
.venv\Scripts\activate
```

Make changes

Each blog post lives in a Quarto file (`.qmd`). It's a mix of markdown and chunk of code.

You can preview locally your changes with:

```
quarto preview
```

If not done automatically, open your browser at <http://localhost:4000/>